

Self-Hosted n8n Setup Documentation

This documents the Docker Compose setup for `n8n.sidshkya.com`, running on Ubuntu via Docker Compose.

1. Architecture Overview

The stack has three containers working together:

- **cloudflared** — Cloudflare Tunnel client. Routes traffic from `n8n.sidshkya.com` into the `n8n` container without needing to open any inbound ports on your router/firewall.
 - **n8n** — The main n8n application: editor UI, REST API, workflow execution engine, and the **task broker** (coordinates work with the runners sidecar).
 - **task-runners** — A separate sidecar container that actually executes user-written JavaScript/Python from **Code nodes**, isolated from the main `n8n` process. This is the secure ("external mode") way to run Code node scripts — see [Task Runners](#) below.
-

2. Services

n8n

Setting	Value
Image	<code>docker.n8n.io/n8nio/n8n:latest</code>
Container name	<code>n8n</code>
Restart policy	<code>unless-stopped</code>
Port	<code>5678</code> (mapped to host)
Data volume	<code>n8n_data:/home/node/.n8n</code>

task-runners

Setting	Value
Image	<code>n8nio/runners:latest</code> (Docker Hub — not <code>docker.n8n.io</code> , that registry doesn't host this image)
Container name	<code>n8n-runners</code>
Restart policy	<code>unless-stopped</code>
Connects to	<code>n8n</code> container's task broker via <code>http://n8n:5679</code>

cloudflared

Setting	Value
Image	cloudflare/cloudflared:latest
Container name	n8n-tunnel
Config	Reads ./cloudflared/config.yml (mounted read-only)

3. Environment Variables Reference

Core n8n settings

Variable	Purpose
N8N_HOST	Public hostname (n8n.sidshkya.com)
N8N_PROTOCOL	https — tells n8n to generate https:// links
N8N_WEBHOOK_URL	Base URL for webhooks. Replaces the deprecated WEBHOOK_URL.
N8N_PORT	Internal port n8n listens on (5678)
N8N_LICENSE_ACTIVATION_KEY	Activates your n8n license/plan
N8N_PROXY_HOPS	Set to 1 since traffic passes through the Cloudflare tunnel (one reverse-proxy hop) — ensures n8n reads the real client IP correctly

Task runner settings (external mode)

Variable	Purpose
N8N_RUNNERS_MODE	external — Code node scripts run in the separate task-runners sidecar, not inside the main n8n process. This is the secure mode; internal mode shares n8n's credentials/DB access with any script.
N8N_RUNNERS_AUTH_TOKEN	Shared secret so n8n and task-runners can authenticate with each other. Must be identical in both services. Stored in .env.
N8N_RUNNERS_BROKER_LISTEN_ADDRESS	0.0.0.0 — lets the broker inside n8n accept connections from the separate task-runners container (default is localhost-only, which wouldn't work across containers)

On the task-runners side:

Variable	Purpose
N8N_RUNNERS_TASK_BROKER_URI	Where to find the broker — http://n8n:5679 (n8n's internal Docker service name + broker port)
N8N_RUNNERS_AUTH_TOKEN	Same shared secret as above

n8n Assistant / AI settings

Variable	Purpose
N8N_ENABLED_MODULES	instance-ai — enables the built-in AI assistant module
N8N_INSTANCE_AI_MODEL	Which model to use via OpenRouter (openrouter/\${OPENROUTER_MODEL})
N8N_INSTANCE_AI_MODEL_API_KEY	OpenRouter API key
N8N_INSTANCE_AI_SANDBOX_ENABLED	true — required for the AI assistant to safely execute anything
N8N_INSTANCE_AI_SANDBOX_PROVIDER	daytona — the sandboxing backend used
N8N_INSTANCE_AI_SANDBOX_IMAGE	Which Daytona sandbox image to use
DAYTONA_API_URL / DAYTONA_API_KEY	Credentials for the Daytona sandbox service
INSTANCE_AI_BRAVE_SEARCH_API_KEY	(Optional) Enables web search for the AI assistant

DNS

```
dns:  
  - 8.8.8.8  
  - 1.1.1.1
```

Overrides the container's default DNS resolver with Google/Cloudflare public DNS — helps avoid resolution failures for workflow nodes that make outbound calls (e.g. Google Sheets triggers).

4. Task Runners & Security

Why this matters: the Code node in n8n lets anyone who can edit a workflow run arbitrary JavaScript/Python. Without isolation, that code runs with the *same* access as n8n itself — meaning it could read your database, credentials, and environment variables.

This setup uses **external mode**: the `task-runners` sidecar executes that code in a separate, isolated container, connected to n8n's task broker only via the shared `N8N_RUNNERS_AUTH_TOKEN`. This is the recommended production configuration.

Keep in mind:

- `N8N_RUNNERS_AUTH_TOKEN` should be treated like a database password — don't commit it to git, keep it in `.env`.
- If you ever need to allow extra JS/Python packages inside Code nodes, that requires extending the `n8nio/runners` image and updating its allowlist config (`n8n-task-runners.json`) — not something configured via this compose file alone.

5. Volumes

Volume	Mounted at	Purpose
<code>n8n_data</code> (external)	<code>/home/node/.n8n</code> inside <code>n8n</code>	Stores the SQLite database, encryption key, and all workflow/credential data. This is your entire n8n instance's state — back it up regularly.

Because it's declared `external: true`, this volume must already exist on the host (it isn't created/destroyed automatically by `docker compose down`), which protects your data from being accidentally wiped.

6. Common Operations

Start everything:

```
docker compose up -d
```

Stop everything:

```
docker compose down
```

(This does *not* delete the `n8n_data` volume since it's external.)

View logs:

```
docker compose logs -f n8n
docker compose logs -f task-runners
docker compose logs -f cloudflared
```

Check n8n's running version:

```
docker exec n8n n8n --version
```

Update to the latest images:

```
docker compose pull
docker compose up -d
```

⚠ Since both `n8n` and `task-runners` are pinned to `:latest`, pulling can change your running version. Check `docker exec n8n n8n --version` before and after to know what changed, and skim n8n's

release notes if it's a major jump.

Back up your data volume:

```
docker run --rm -v n8n_data:/data -v $(pwd):/backup alpine \
  tar czf /backup/n8n_data_backup_$(date +%Y%m%d).tar.gz -C /data .
```

7. Known Gotchas (from setting this up)

1. **docker.n8n.io/n8nio/runners does not exist** — the runners image is only published on Docker Hub as **n8nio/runners**, without the **docker.n8n.io/** prefix that the main **n8n** image uses.
2. **Exact-version tags aren't guaranteed to match between n8n and runners** — the runners image doesn't get a tag for every n8n patch release. Using **:latest** for both keeps them in sync automatically; pinning an exact version requires first confirming that tag exists for *both* images on Docker Hub.
3. **WEBHOOK_URL is deprecated** — use **N8N_WEBHOOK_URL** instead. Both env vars work today, but only set one.
4. **Transient DNS errors on startup** — a workflow node occasionally logs **The DNS server returned an error, perhaps the server is offline** right at container boot, before networking is fully up. It has self-resolved on retry in this setup and hasn't recurred as a persistent issue.

8. .env File Checklist

Your **.env** (not committed to git) should contain:

```
N8N_LICENSE_ACTIVATION_KEY=...
N8N_RUNNERS_AUTH_TOKEN=...      # generate via: openssl rand -hex 32
OPENROUTER_MODEL=...
OPENROUTER_API_KEY=...
DAYTONA_API_KEY=...
BRAVE_SEARCH_API_KEY=...
```